

WEB DEVELOPMENT TRACK • SESSION 06 (MILESTONE)

Phase 1 Capstone Project & Introduction to Version Control (Git)

Building a Multi-Page Semantic Website & Mastering git init, git add, git commit

Instructor Edition | End of HTML5 Milestone & Git Basics

Capstone Project: Multi-Page Tribute / Portfolio

Project Objectives

Combine all HTML tags learned across Weeks 1 to 3.

Pure Semantic HTML: No external CSS frameworks yet.

Multi-page architecture linked via clean relative links.

Clean folder structure with dedicated asset management.

💡 Purpose: Transforms isolated exercises into a portfolio-ready standalone website.

Required Multi-Page Architecture

1. `index.html` (Home / Bio / Tribute Intro)
2. `achievements.html` (Tables, Timelines & Milestones)
3. `contact.html` (Full HTML5 Interactive Contact Form)
4. Shared Nav: `<nav>` containing relative links to all 3 pages

Semantic Skeleton: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`

Why Developers Need Git (The Time Machine)

The Beginner Nightmare (No Git)

project_final.html

project_final_final.html

project_final_copy(1)_REAL.html

- No record of WHO changed WHAT or WHEN.
- If code breaks, there is no 'Undo' from yesterday.
- Impossible to collaborate seamlessly in a team.

The Git Solution (Version Control)

- A distributed Version Control System (VCS).
- Takes 'Snapshots' (Commits) of your project over time.
- Time Machine: Revert back to any previous working state anytime.
- Git is LOCAL (runs on your machine).
- GitHub is REMOTE (the cloud host for Git repositories).

The 3 Stages of Git Workflow

1. Working Directory

Your Local Files

Files currently being edited in VS Code. Changes here are 'Untracked' or 'Modified'.

Status Check:
`$ git status`

2. Staging Area

The Loading Dock

Files marked ready for a snapshot via 'git add'. A deliberate prep area before permanent saving.

Command:
`$ git add <file>`

3. Git Repository

Permanent Vault

Snapshots safely recorded in the local .git directory via 'git commit'. Permanent history.

Command:
`$ git commit -m '...'`

Git Setup & Repository Initialization

One-Time Global Configuration

Run these commands once when installing Git:

```
$ git --version
```

```
$ git config --global user.name "Student Name"
```

```
$ git config --global user.email  
"email@test.com"
```

Initializing a Repo: git init

- Transforms any ordinary project folder into a tracked Git repository.
- Creates a hidden '.git' tracking folder in the background.
- Do NOT touch or delete the .git folder manually!

```
$ cd my-portfolio
```

```
$ git init
```

Initialized empty Git repository in .../.git/

The Core Loop: git status, add & commit

Staging Changes (git add)

- `git status` : Checks which files are modified, untracked (U), or staged (A).
- `git add <filename>` : Stages a specific file (clean granular history).
- `git add .` : Stages all modified and new files at once.

```
$ git status
$ git add index.html
$ git status
```

Saving Snapshots (git commit)

- `-m` flag: Provides a clear, descriptive message explaining the snapshot.
- Best Practice: Use present-tense imperative verbs ('Add', 'Fix', 'Update').
- `git log` : Shows the chronological history of commits with author & date.

```
$ git commit -m "feat: create home page
skeleton"
[main (root-commit) 8a1b2c] feat: create ...

$ git log --oneline
```

Hands-On Pairing: Capstone Kickoff & First Repo

Step 1: Project Scaffolding

Create project folder 'capstone-portfolio' with index.html, achievements.html, and contact.html.

⚡ Folder structure & boilerplates

Step 2: Git Repository Setup

Open terminal in project folder, run 'git init', and verify the tracking state with 'git status'.

⚡ Initialize local repository

Step 3: Build & Commit Index

Code semantic <header>, <nav>, and <main> bio in index.html, then run 'git add index.html' and commit.

⚡ First recorded commit

Step 4: Build Contact & Second Commit

Implement contact.html with a full HTML5 form, stage it, and commit with a clean message.

⚡ Second recorded commit

Phase 1 Capstone Deliverables (Due Next Week)

HTML Technical Rubric

- 3 linked pages (index, achievements, contact).
- Semantic layout: header, nav, main, article, section, footer.
- achievements.html: Structured table with thead, tbody, tfoot, colspan, rowspan.
- contact.html: Accessible form with labels, fieldset, legend, diverse input types.
- Proper alt text on all images, and target='_blank' on external links.

Git Version Control Rubric

- Project folder initialized with git init.
- Minimum 4 distinct, meaningful commits.
- Clear commit messages following standard conventions.
- Submission: Zipped project folder including the hidden .git directory (or git log screenshot).

 Phase 1 (Pure HTML) Complete! CSS begins next weekend.